

Hibernate ID Generation Strategies

A simple guide to AUTO, IDENTITY, SEQUENCE, and TABLE

Objective: Understand what each ID generation strategy does, when to use it, and how to choose the right one instead of only memorizing the names.

1. GenerationType.AUTO

1. What is it?

Hibernate/JPA automatically chooses an ID generation strategy based on the database and JPA provider.

2. When to use it?

Use it when you do not need to control exactly how IDs are generated and you want a simple, portable option.

3. Example

```
@Id
@GeneratedValue(strategy = GenerationType.AUTO)
private Long id;
```

4. Database

Suitable for many databases, such as PostgreSQL, MySQL, Oracle, and H2. The actual mechanism chosen can differ.

2. GenerationType.IDENTITY

1. What is it?

The database generates the ID using an identity or auto-increment column. The ID is normally created when the row is inserted.

2. When to use it?

Use it when your database has built-in auto-increment or identity columns and you want the database to handle ID creation directly.

3. Example

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

4. Database

Common with MySQL and SQL Server. PostgreSQL also supports identity columns.

3. GenerationType.SEQUENCE

1. What is it?

The ID is generated from a database sequence. Hibernate can request the next sequence value before inserting the row.

2. When to use it?

Use it when the database supports sequences. It is often a strong choice because Hibernate can optimize ID allocation and batching.

3. Example

```
@Id
@GeneratedValue(
    strategy = GenerationType.SEQUENCE,
    generator = "student_seq"
)
@SequenceGenerator(
    name = "student_seq",
    sequenceName = "student_sequence",
    allocationSize = 1
)
private Long id;
```

4. Database

Good for PostgreSQL and Oracle. It requires a database that supports sequences.

4. GenerationType.TABLE

1. What is it?

A separate database table stores and manages the next ID value. Hibernate reads and updates that table to generate IDs.

2. When to use it?

Use it mainly when you need a database-independent solution and cannot use identity columns or sequences. It is usually slower because it needs extra table access.

3. Example

```
@Id
@GeneratedValue(
    strategy = GenerationType.TABLE,
    generator = "id_gen"
)
@TableGenerator(
    name = "id_gen",
    table = "id_generator",
    pkColumnName = "gen_name",
    valueColumnName = "gen_value",
    allocationSize = 1
)
private Long id;
```

4. Database

Can work with almost any relational database because it uses a normal table.

Simple Comparison

Strategy	Who generates the ID?	Best use	Example DB
AUTO	Hibernate/JPA chooses	Simple default and portability	Many DBs
IDENTITY	Database auto-increment	Databases with identity columns	MySQL, SQL Server
SEQUENCE	Database sequence	Good performance with sequence-supporting DBs	PostgreSQL, Oracle
TABLE	Special ID table	Fallback with broad DB compatibility	Almost any DB

Quick Way to Remember

AUTO = let Hibernate choose.

IDENTITY = database auto-increment.

SEQUENCE = database sequence generates numbers.

TABLE = a separate table keeps the next ID.

Which should you choose? For learning, AUTO is the easiest starting point. For MySQL, IDENTITY is common. For PostgreSQL or Oracle, SEQUENCE is often preferred. TABLE is usually a fallback when the other mechanisms are not suitable.

Note: These are JPA generation strategies used by Hibernate as a JPA provider. Exact behavior can vary slightly depending on the database and Hibernate version.